

나의 영토 도메인 분석 · 애그리거트 설계

Oct 2, 2026 · @https://m.kinolights.com/ranking/netflix

서비스 개요와 유비쿼터스 언어

「나의 영토」는 다녀온 시·군·구를 지도에 칠하면 특산물 아이템과 풍경을 얻어 캐릭터·장면을 꾸미고, 그 결과를 친구에게 자랑하는 여행 기록 서비스다. 핵심 도메인은 **탐험(Exploration)** 하나이고, 나머지는 탐험 기록을 소비하는 지원·일반 도메인이다. 돈이 오가지 않고 모든 입력이 자기 신고제라는 점이 설계 전체의 전제다.

용어	정의	영문 식별자
지역	행정구역 시·군·구 단위의 칠할 수 있는 최소 영역. 250개 고정	Region
시·도	지역의 상위 묶음(17개). 정복률 집계 단위	Province
희귀도	지역의 등급. 일반(시·구) · 희귀(군) · 전설(지정 10 곳)	Rarity
발 도장 / 체크인	특정 지역을 "다녀왔다"고 기록하는 행위. 날짜·메모·사진을 가진다	Visit (Check-in)
영토	한 탐험가가 체크인한 지역의 집합	Territory
정복률	영토 ÷ 전체 지역 (전국·시·도별)	ConquestRate
탐험가	서비스 사용자. 레벨·칭호·XP를 가진다	Explorer
XP / 레벨 / 칭호	체크인·보너스로 쌓이는 점수와 그 파생값	Xp, Level, Title
스트릭	매달 새 지역 1곳 이상 체크인한 연속 개월 수	Streak
뱃지	영구 획득 업적. 한번 얻으면 회수되지 않는다	Badge
테마 도감 / 세트	테마로 묶인 지역 목록. 전부 모으면 완성	CollectionSet
월간 퀘스트 / 상시 도전	기간 한정·누적형 과제. 완료 시 XP·칭호	Quest, Challenge
특산물 아이템	지역 체크인 시 1:1로 주어지는 꾸미기 아이템. 슬롯 (모자·손·키링·배낭·동행·배경·장식)을 가진다	Item
전설 풍경	전설 지역 아이템. 배경 슬롯에 들어가는 지역 고유 풍경	LegendView
세트 배경	세트 완성 보상으로 열리는 프리미엄 배경	SetBackground

용어	정의	영문 식별자
장면	캐릭터(성별)+착용 아이템+배경+장식의 현재 조합	Scene (Avatar)
꾸미기 점수	착용 아이템 희귀도 점수의 합	StylePoints
자랑 카드	영토·최근 여행·리캡·VS 비교를 이미지로 만든 공유물. 공개 프로필의 OG 이미지	ShareCard
친구 / 영토 비교	팔로우 관계와 두 영토의 집합 연산(나만·둘 다·상대만)	Friendship, TerritoryDiff

바운디드 컨텍스트

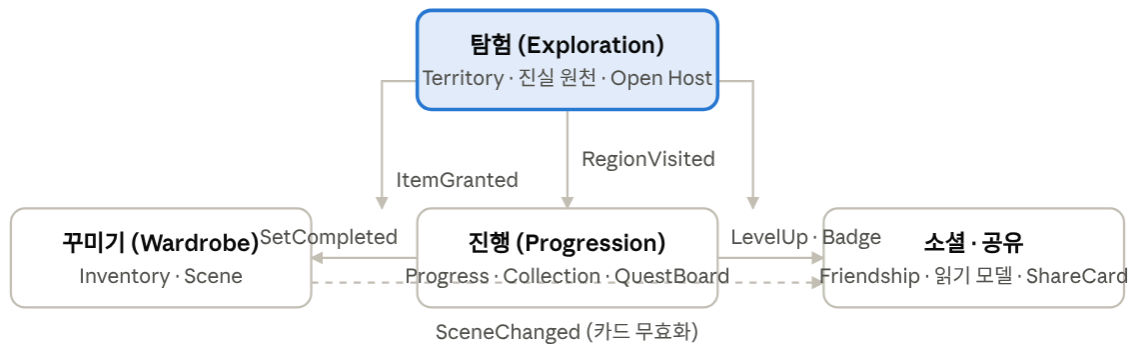
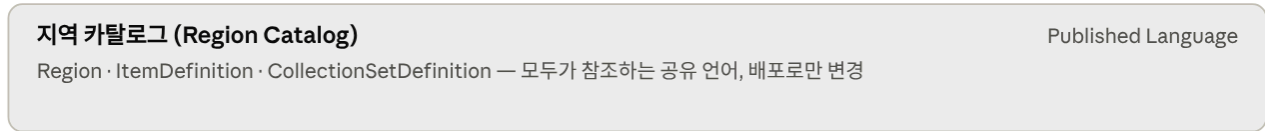
컨텍스트는 6개로 나뉘고, 탐험 컨텍스트가 내는 이벤트를 나머지가 구독하는 구조다. 체크인 한 번이 진행·도감·인벤토리·소셜을 연쇄로 움직이지만, 트랜잭션은 탐험 컨텍스트 안에서만 닫는다.

컨텍스트	책임	주요 애그리거트	일관성
지역 카탈로그 (Region Catalog)	시·군·구 기준 데이터, 희귀도, 특산물 정의, 테마 세트 정의, 경계 GeoJSON	Region, CollectionSetDefinition	정적 참조 데이터. 배포 단위로만 변경
탐험 (Exploration)	체크인·취소, 방문 기록 (날짜·메모·사진), 영토 집합	Territory	강한 일관성. 시스템의 진실 원천 (source of truth)
진행 (Progression)	XP·레벨·칭호·스트릭·배지·퀘스트·도감 완성 판정	ExplorerProgress, QuestBoard, Collection	이벤트 기반 최종 일관성. 영토로부터 재계산 가능
꾸미기 (Wardrobe)	아이템 보유, 슬롯 착용, 장면 구성, 꾸미기 점수	Inventory, Scene	최종 일관성. 보유 목록은 영토의 파생 집합
소셜 (Social)	친구 관계, 랭킹, 영토 비교, 활동 피드	Friendship, ActivityFeed, Leaderboard(읽기 모델)	최종 일관성. 집계는 배치·캐시
공유 (Sharing)	공개 프로필, 자랑 카드 렌더링, OG 이미지	PublicProfile(읽기 모델), ShareCard	상태 없음에 가까움. 스냅샷 캐시

상류→하류 관계는 다음과 같다. 지역 카탈로그는 모든 컨텍스트의 상류(Published Language)다. 탐험은 진행·꾸미기·소셜·공유의 상류이고 이벤트로만 말한다(Open Host). 진행은 꾸미기의 상류다(세트 완성 → 세트 배경 해금). 소셜과 공유는 순수 하류로, 이벤트를 읽어 자기 읽기 모델을 갱신할 뿐 상류를

바꾸지 못한다.

지역 카탈로그와 탐험이 상류, 소셜과 공유는 읽기만 하는 하류



화살표 = 이벤트 방향(상류 → 하류). 점선은 읽기 전용 투영.

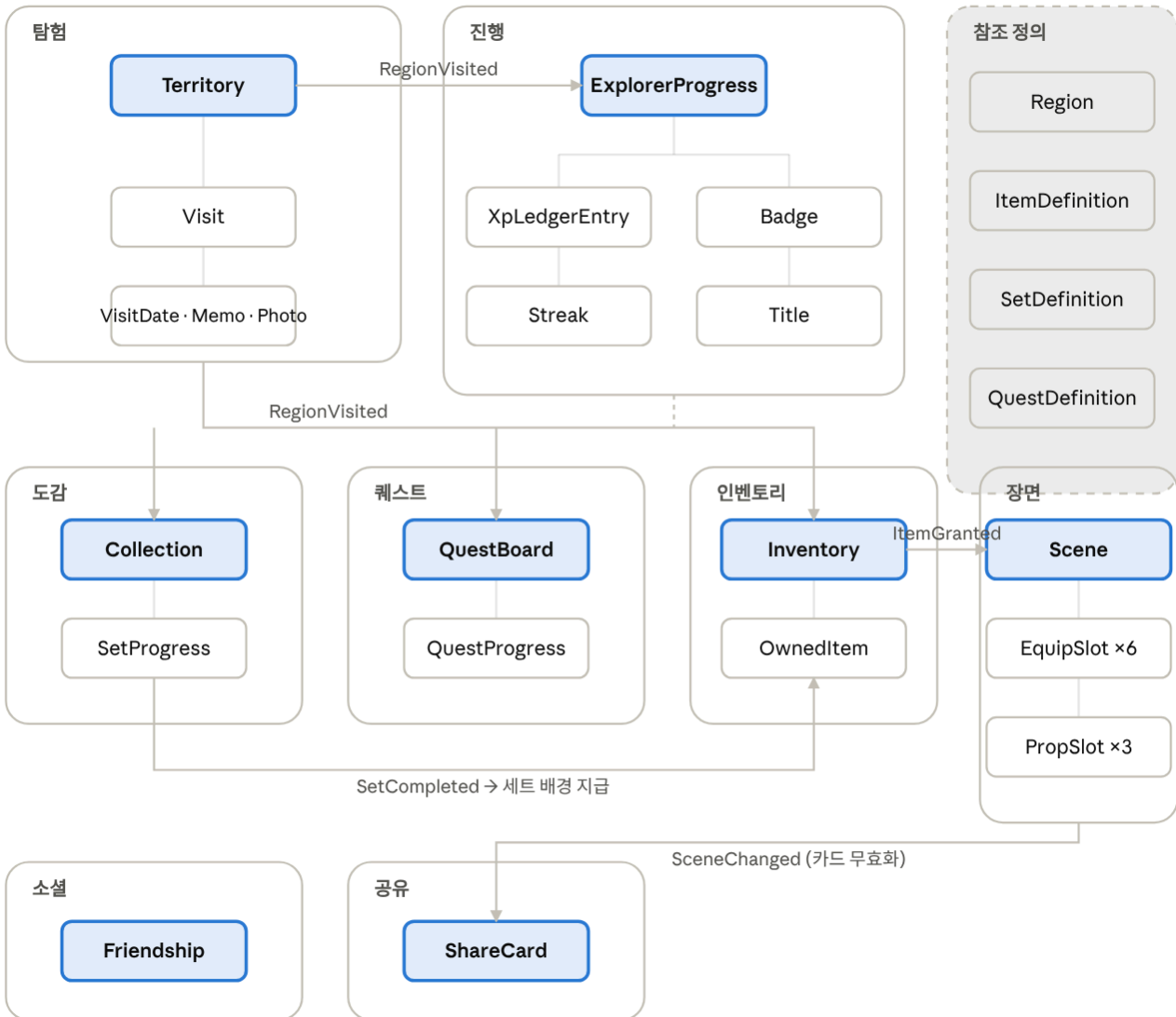
컨텍스트 맵 · 6개 컨텍스트, 상류→하류

화살표는 이벤트가 흐르는 방향이고, 기준 데이터는 마지막 단계에서도 참조된다.

애그리거트

애그리거트는 9개다. 기준은 "한 트랜잭션에서 함께 지켜야 하는 불변식이 있는가"이고, 이 기준으로 보면 탐험가 한 명의 데이터를 하나의 거대 애그리거트로 묶을 이유가 없다. 영토만 강한 일관성이고 나머지는 이벤트로 따라간다. 영토와 도감은 탐험가가 아니라 **공유 지도(ExpeditionMap)** 단위였다(초대코드로 같은 지도를 여러 명이 칠한다). 진행·퀘스트·인벤토리·장면은 탐험가 단위다.

애그리거트 8개: 묶음 안은 한 트랜잭션, 묶음 사이는 이벤트



채워진 상자 = 루트, 연한 선 = 같은 애그리거트 안 구성, 화살표 = 이벤트, 점선 묶음 = 코드로만 참조하는 정의.

애그리거트 구성도 · 묶음 8개 + 참조 정의

묶음 하나가 애그리거트 하나다. 묶음 안의 선은 루트가 필드로 들고 있는 엔티티·VO이고, 묶음 사이는 화살표(이벤트)로만 이어진다. 도감·퀘스트·인벤토리는 모두 탐험에서 같은 이벤트를 받지만 서로를 모른다. 오른쪽 점선 묶음은 애그리거트가 아니라 코드로 참조하는 정의다.

1. Territory (탐험 · 핵심)

루트는 Territory(mapId), 내부에 Visit 엔티티 컬렉션(regionCode → Visit)을 가진다. 지도당 하나, 국가당 최대 지역 수(한국 250)만큼이라 한 덩어리로 로드해도 가볍다. Visit은 누가 체크인했는지 checkedInBy를 가지고, verification은 지금은 항상 NONE이다.

- VO: RegionCode, VisitDate, Memo(≤40자), PhotoRef
- 불변식: 같은 지역은 한 번만 방문 상태(중복 불가). 방문일은 오늘 이후 불가. 취소는 방문 상태에서에만.

- 커맨드: `checkIn(region, date, memo)`, `editVisit(region, date, memo, photo)`, `cancelVisit(region)`
- 이벤트: `RegionVisited{explorerId, regionCode, rarity, provinceCode, visitedAt, isFirstInProvince, nth}`, `VisitEdited`, `VisitCancelled`
- 주의: `isFirstInProvince` 와 `nth` 는 탐험 애그리거트가 자기 상태로 계산해 이벤트에 실어 보낸다. 하루가 영토를 다시 읽지 않게 하기 위해서다.

체크인 규칙(Territory 불변식 추가). 방문일은 과거 날짜 허용, 미래 날짜 거부. 하루 상한은 지도별·멤버별 5건이며, 지도 가입 후 72시간은 온보딩 예외로 상한을 적용하지 않는다(온보딩 첫 화면 = "다녀온 곳 한번에 찍기"). 지도 설정이 `photoRequired` 이면 사진 없는 체크인은 거부한다. 지도장은 특정 방문에 `disputed` 플래그를 세울 수 있고, 플래그가 선 방문은 지도 내 랭킹 집계에서 제외된다(전체 랭킹·개인 영토에는 영향 없음).

2. ExplorerProgress (진행)

루트는 `ExplorerProgress(explorerId)`. XP 장부, 레벨, 선택 칭호, 스트릭, 획득 배지 목록을 가진다.

- VO: `XpLedgerEntry{source, amount, refId, at}`, `Level`, `Title`, `Streak{months, lastMonth}`, `BadgeId`
- 불변식: XP는 장부 합계와 항상 같다(음수 항목으로만 감소). 레벨은 XP의 결정적 함수 ($\text{floor}((1+V(1+xp/5))/2)$). 배지는 추가만 되고 회수 안 됨. 선택 칭호는 획득한 것 중에서만.
- 커맨드: `applyVisit(RegionVisited)`, `revokeVisit(VisitCancelled)`, `grantBadge`, `selectTitle`, `grantQuestReward`
- 이벤트: `XpGained`, `LevelUp`, `BadgeEarned`, `StreakChanged`
- 장부 항목의 `refId` 로 방문 취소 시 해당 XP만 정확히 되돌린다.

3. Collection (도감 진행)

루트는 `Collection(explorerId)`. 세트별 진행 상태와 완성 여부를 가진다.

- VO: `SetProgress{setId, collected:Set<RegionCode>, completedAt?}`
- 불변식: 완성은 정의된 지역을 전부 모았을 때 단 한 번만 발생. 완성 후 지역을 취소해도 완성 기록은 유지(보상 회수 안 함).
- 커맨드: `applyVisit`, `revokeVisit`
- 이벤트: `SetProgressed{setId, have, total}`, `SetCompleted{setId}` → 진행(보너스 XP)과 꾸미기(세트 배경 지급)이 구독
- `ExplorerProgress`와 분리한 이유: 세트 정의가 추가될 때마다 진행 루트가 커지는 것을 막고, "전체 재계산" 배치를 독립 실행하기 위해서다.

4. QuestBoard (월간 퀘스트 · 상시 도전)

루트는 `QuestBoard(explorerId, yearMonth)`. 달이 바뀌면 새 보드가 생긴다. 상시 도전은 `yearMonth = null` 인 보드 하나로 다룬다.

- VO: `QuestProgress{questId, current, target, claimedAt?}`
- 불변식: 보상은 달성 후 한 번만 지급. 지난 달 보드는 불변.
- 커맨드: `applyVisit, claim(questId)`
- 이벤트: `QuestCompleted{questId, xp}` → 진행이 구독
- 스트릭은 여기가 아니라 `ExplorerProgress`에 둔다. 스트릭은 달을 넘나드는 연속 값이고 퀘스트는 달로 닫히기 때문이다.

5. Inventory (보유 아이템)

루트는 `Inventory(explorerId)`. 보유 아이템 집합과 획득 시각을 가진다.

- VO: `OwnedItem{itemId, source(REGION|SET_REWARD|EVENT), acquiredAt}`, `ItemId` 는 지역 아이템이면 `region:{code}`, 세트 보상이면 `set:{setId}`
- 불변식: 같은 `itemId`는 한 번만. 지역 아이템은 해당 지역 방문 취소 시 회수. 세트 보상은 회수 안 함.
- 커맨드: `grant(itemId, source), revoke(itemId)`
- 이벤트: `ItemGranted, ItemRevoked` → `Scene`이 구독(착용 중인 것이 회수되면 벗김)
- 아이템의 생김새(슬롯·룩·색·이름)는 여기 없다. 지역 카탈로그의 `ItemDefinition` 을 `itemId`로 참조한다.

6. Scene (장면 · 꾸미기)

루트는 `Scene(explorerId)`. 성별·슬롯별 착용 아이템·장식 3칸을 가진다.

- VO: `Gender, EquipSlot(HAT|HAND|BADGE|BAG|PET|BG), PropSlots(≤3)`
- 불변식: 착용 아이템은 모두 `Inventory`에 있어야 한다. 한 슬롯에 하나, 장식은 3개까지, 아이템 슬롯과 장착 슬롯이 일치.
- 커맨드: `equip(slot, itemId), unequip(slot), setGender, autoEquip(itemId)` (빈 슬롯이거나 더 높은 희귀도일 때)
- 이벤트: `SceneChanged` → 공유가 구독(카드 스냅샷 무효화)
- `Inventory`와 분리한 이유: 보유는 탐험에서 파생되고, 착용은 사용자 선택이다. 변경 빈도와 주체가 다르다.

7. Friendship (소셜)

루트는 `Friendship(fromExplorerId, toExplorerId)`. 팔로우 관계 한 건이 한 애그리거트다.

- 불변식: 자기 자신 팔로우 불가, 중복 불가

- 커맨드: `follow`, `unfollow`
- 랭킹·VS 비교·피드는 애그리거트가 아니라 읽기 모델이다.
`RegionVisited` · `SetCompleted` · `LevelUp` 을 구독해 `ActivityFeedEntry` 와 `LeaderboardRow` 를 쌓는다.

8. ShareCard (공유)

루트는 `ShareCard(explorerId, kind, version)`. 종류는 영토·최근 여행·리캡·VS.

- 불변식: 참조한 영토·장면 버전이 기록되고, 그 버전이 바뀌면 재생성
- 커맨드: `render()`, `invalidate()`
- 사실상 캐시에 가까운 애그리거트다. 공개 프로필 페이지와 OG 이미지는 여기서 나간다.

9. ExpeditionMap (공유 지도)

루트는 `ExpeditionMap(mapId)`. 이름, 초대코드, 생성자, 멤버 목록, 국가 코드를 가진다. 가입 시 개인 지도가 자동 생성된다.

- VO: `InviteCode`(8자, 재발급 가능), `Member{explorerId, role(OWNER|MEMBER), joinedAt}`, `CountryCode`
- 불변식: 멤버 ≤4. 초대코드는 지도당 하나이고 전체 유일. OWNER는 한 명, 탈퇴 불가(양도 후 가능). 같은 탐험가 중복 가입 불가.
- 커맨드: `create(owner, name, country)`, `join(inviteCode, explorerId)`, `leave`, `transferOwner`, `regenerateInviteCode`
- 이벤트: `MapCreated` → 탐험이 빈 `Territory`와 `Collection`을 만든다. `MemberJoined` → 인벤토리가 이미 완성된 세트 보상을 새 멤버에게 지급한다
- `Territory`와 분리한 이유: 멤버 가입·탈퇴가 250건 방문 기록과 같은 락을 잡을 이유가 없다. 체크인은 "이 탐험가가 이 지도의 멤버인가"만 조회해 확인한다(애그리거트 간 읽기 참조).

지도 설정과 탈퇴 유예. `ExpeditionMap`은 `MapSettings` 값 객체(`photoRequired`, `dailyCheckInCap` 기본 5, `visibility`)를 갖고, 생성 시 멤버에게 3줄 규칙(선점 보너스·사진 여부·하루 상한)을 확인시킨다. 탈퇴는 7일 소프트 삭제다: 탈퇴 즉시 해당 멤버의 방문에 `hidden_at` 을 찍고 선점(claim)은 다음 체크인 멤버에게 이전(`ClaimTransferred`)한다. 7일 안에 재가입하면 숨긴 방문은 복구되지만 이전된 선점은 돌아오지 않는다. 7일 후 배치가 하드 삭제한다.

참조 데이터 (애그리거트 아님)

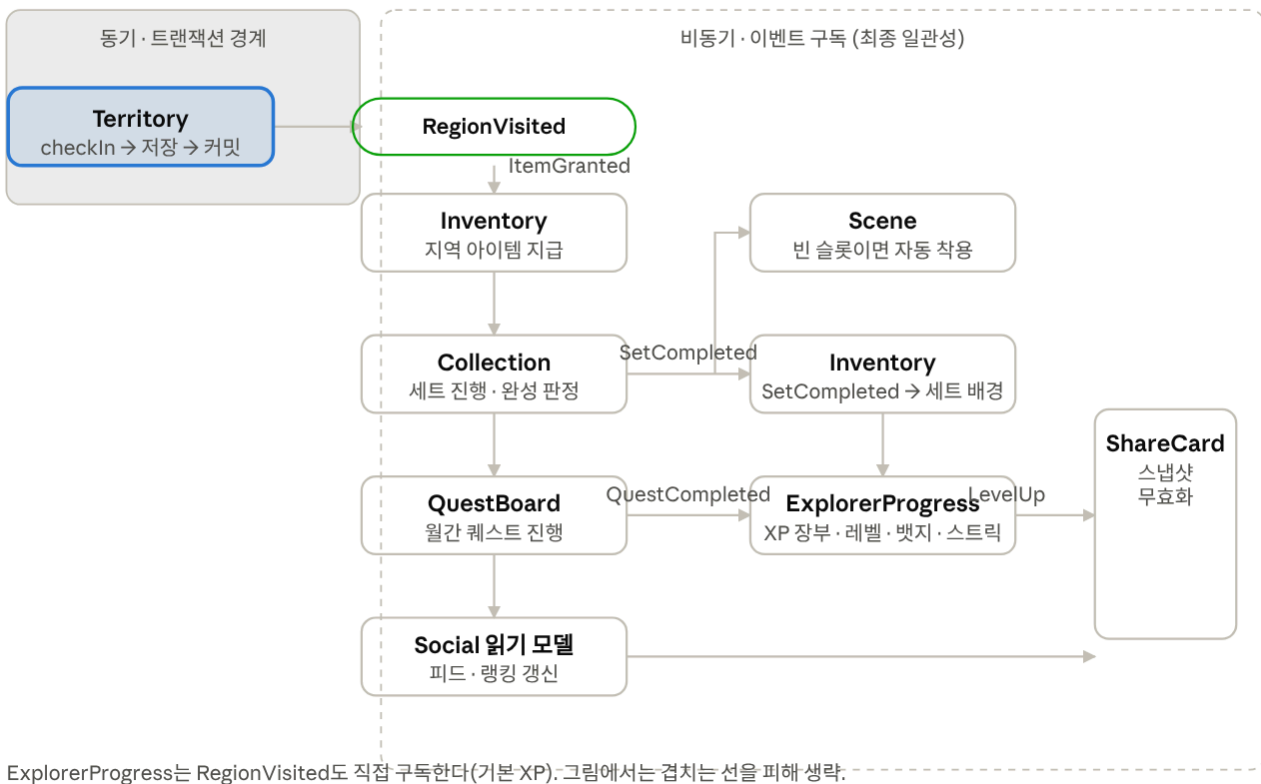
`Region{code, name, province, rarity, geometry}`, `ItemDefinition{itemId, slot, look, colors, name, tier}`, `CollectionSetDefinition{setId, name, regionCodes, rewardTitle, rewardBackground}`, `BadgeDefinition`, `QuestDefinition` 은 배포 단위로 바뀜은 정적 데이터다. DB 테이블보다 코드·리소스 파일로 두고 시작 시 메모리에 올리는 편이 맞다.

애그리거트	크기(탐험가당)	변경 주체	이벤트 소비
Territory	≤250 Visit	사용자	—
ExplorerProgress	XP 장부 + 배지	이벤트	RegionVisited, VisitCancelled, SetCompleted, QuestCompleted
Collection	세트 9개 진 행	이벤트	RegionVisited, VisitCancelled
QuestBoard	월별 4건 + 상시	이벤트 + 사용자 (claim)	RegionVisited
Inventory	≤260 아이 템	이벤트	RegionVisited, VisitCancelled, SetCompleted
Scene	슬롯 6 + 장 식 3	사용자	ItemGranted(자동 착용), ItemRevoked
Friendship	관계 1건	사용자	—
ShareCard	종류별 1건	시스템	SceneChanged, RegionVisited

체크인 플로우와 일관성 경계

트랜잭션은 Territory 하나만 잠그고, `RegionVisited` 이벤트를 같은 트랜잭션의 outbox에 쌓는다. 나머지 애그리거트는 이벤트를 구독해 각자 자기 트랜잭션에서 갱신하고, 순서가 중요한 것(세트 완성 → 보상)은 2차 이벤트로 이어진다.

체크인 한 번은 트랜잭션 하나로 닫히고, 나머지는 이벤트로 따라간다



체크인 이벤트 흐름 · 애그리거트 8개 중 7개 관여

프로토타입에서는 이 전부가 클릭 한 번에 동기로 돌지만, 서버에서는 체크인 응답에 필요한 것(영토 반영, 받을 아이템, 예상 XP)만 동기로 계산해 돌려주고 실제 반영은 비동기로 두는 게 맞다. 체크인 modal의 "획득 XP +90" 같은 숫자는 도메인 서비스 `CheckInPreview` 가 순수 함수로 계산한다(현재 영토 + 지역 → 보상 목록). 같은 함수를 이벤트 핸들러가 다시 써서 예상과 실재가 어긋나지 않게 한다.

일관성 규칙은 세 가지다. 첫째, 모든 핸들러는 멍등하다(`refId` 로 중복 적용을 막는다). 둘째, 취소는 보상 중 지역 아이템과 기본 XP만 되돌리고 세트 완성·배지·퀘스트 보상은 유지한다(난이도 대비 악용 가치가 낮고 사용자 경험이 나쁘다). 셋째, 진행·도감·인벤토리는 Territory로부터 전부 재계산할 수 있어야 한다(정의 변경·버그 복구용 배치).

Spring Boot 구현 매핑

바운디드 컨텍스트 하나를 Gradle 모듈 하나로 두는 멀티모듈 구조다. 실행 가능한 Spring Boot 앱은 `app-api` 하나뿐이고, 컨텍스트 모듈은 서로의 `api` 패키지(이벤트·조회 인터페이스)만 볼 수 있다. 컨텍스트 간 호출은 `ApplicationEventPublisher` + `outbox`로만 하고, Kafka는 트래픽이 생긴 뒤 `outbox` 릴레이로 붙인다.

```
territory/                                settings.gradle.kts - include("common",
"catalog", "exploration", ..., "app-api")
├─ common/                               공유 커널: ExplorerId, RegionCode, DomainEvent, Outbox,
Clock. Spring 의존 없음
├─ catalog/                             Region, ItemDefinition, CollectionSetDefinition 로더 (리소스
JSON → 메모리)
├─ exploration/                         api/ RegionVisited, VisitCancelled, TerritoryQuery
|                                       domain/ Territory, Visit, CheckInPreview
|                                       application/ CheckInService
|                                       infra/ JpaTerritoryRepository, VisitEntity
├─ progression/                         api/ SetCompleted, QuestCompleted, LevelUp, ProgressQuery
|                                       domain/ ExplorerProgress, Collection, QuestBoard
|                                       application/ 이벤트 핸들러, RecalculateService
|                                       infra/ JPA 엔티티·리포지토리
├─ wardrobe/                           api/ ItemGranted, SceneChanged, SceneQuery · domain/
Inventory, Scene
├─ social/                             api/ FeedQuery, LeaderboardQuery · domain/ Friendship ·
application/ 프로젝트
├─ sharing/                             ShareCardRenderer, PublicProfileQuery (다른 컨텍스트의 Query
만 의존)
└─ app-api/                             @SpringBootApplication, REST 컨트롤러, 보안, Flyway, outbox
릴레이 스케줄러
```

모듈	의존 허용	금지
common	없음	Spring, JPA
catalog	common	다른 컨텍스트
exploration	common, catalog	progression·wardrobe·social·sharing
progression	common, catalog, exploration:api	exploration의 domain/infra
wardrobe	common, catalog, exploration:api, progression:api	—
social	common, exploration:api, progression:api	다른 컨텍스트의 domain
sharing	common, catalog, 모든 컨텍스트의 api(Query)	다른 컨텍스트의 domain
app-api	전부	도메인 로직 작성

:api 경계는 Gradle로는 강제가 안 되므로(모듈 전체가 클래스패스에 오른다) ArchUnit 테스트 한 개로 막는다: " ..exploration.. 패키지는 ..exploration.api.. 를 제외하고 다른 컨텍스트에서 참조할 수 없다". 정말 단단히 막으려면 컨텍스트마다 exploration-api · exploration-core 두 모

둘로 쪼개면 되지만, 모듈 수가 두 배가 되므로 팀이 커지기 전까지는 ArchUnit으로 충분하다.

각 컨텍스트 모듈은 `domain` 에 Spring·JPA 어노테이션을 두지 않고 순수 Java로 둔다. JPA 엔티티는 `infra` 에 따로 두고 리포지토리가 변환한다. 이렇게 하면 Territory 같은 루트는 단위 테스트가 컴텍스트 없이 돌고, 나중에 특정 컨텍스트만 별도 서비스로 떼어낼 때 모듈 하나를 그대로 가져가면 된다.

테이블	애그리거트	키	비고
<code>visit</code>	Territory	(explorer_id, region_code) UNIQUE	날짜·메모·사진 URL·생성시각. 취소는 물리 삭제 + 이벤트
<code>explorer_progress</code>	ExplorerProgress	explorer_id	xp, level, title_id, streak_months, streak_last_month, version(낙관적 락)
<code>xp_ledger</code>	ExplorerProgress	id	source, amount, ref_id UNIQUE(먹등), created_at
<code>badge_earned</code>	ExplorerProgress	(explorer_id, badge_id)	earned_at
<code>set_progress</code>	Collection	(explorer_id, set_id)	collected_codes JSON, completed_at
<code>quest_progress</code>	QuestBoard	(explorer_id, year_month, quest_id)	current, claimed_at
<code>owned_item</code>	Inventory	(explorer_id, item_id)	source, acquired_at
<code>scene</code>	Scene	explorer_id	gender, slots JSON, props JSON, version
<code>friendship</code>	Friendship	(from_id, to_id)	created_at
<code>feed_entry</code>	읽기 모델	id	actor_id, kind, payload JSON, created_at. 인덱스 (actor_id, created_at)
<code>outbox</code>	공통	id	aggregate, event_type, payload JSON, published_at

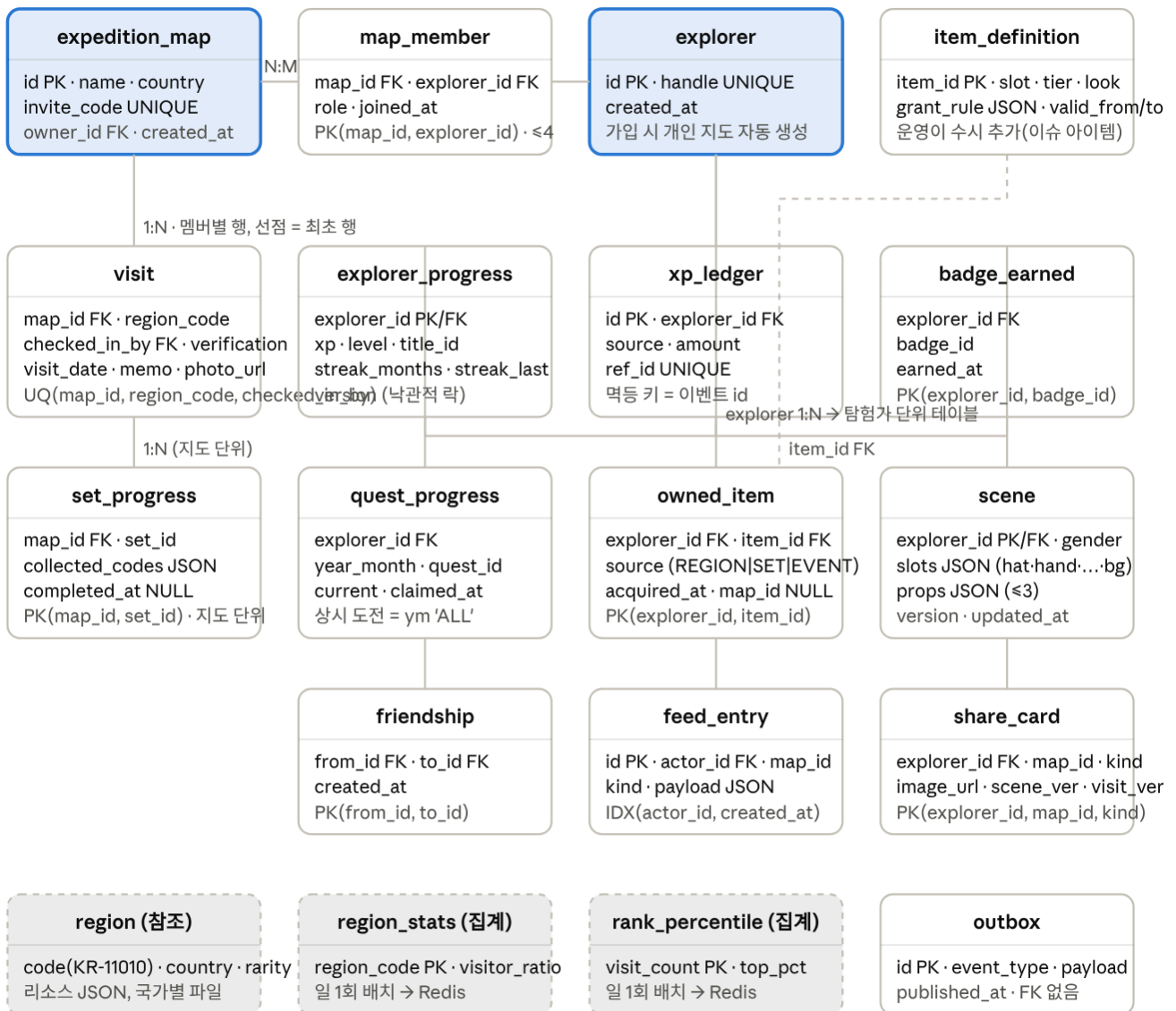
집계와 캐시는 다음처럼 둔다. 시·도별 정복률과 전국 정복률은 Territory를 읽을 때 메모리에서 계산한다(250건). 지역별 방문자 비율(희귀도 표시용)과 전체 유저 중 상위 N%는 하루 1회 배치로 `region_stats · rank_percentile` 테이블에 쌓고 Redis에 올린다. 친구 랭킹은 친구 수가 작으므로 요청 시점에 `explorer_progress` 와 `visit` 카운트를 조인해 계산한다. 자랑 카드는 생성 후 오브젝트 스토리지에 두고 `SceneChanged · RegionVisited` 에서 무효화한다.

지역·아이템·세트 정의는 현재 프로토타입의 JS 객체를 JSON 리소스로 그대로 옮기면 된다. 아이템의 슬롯·룩 타입·색·이름은 클라이언트 렌더링용 정보이므로 서버는 `itemId` 와 `tier` 만 알면 되고, 렌더링 룩은 클라이언트가 같은 카탈로그를 들고 있다.

ERD

테이블은 14개고 루트는 둘(지도·탐험가)이다. 영토·도감은 `expedition_map` 으로, 나머지 FK는 전부 `explorer` 로만 향한다. 애그리거트 구성도와 맞춰 다시 점검하면서 세 가지를 고쳤다. `gender` 는 Scene 애그리거트가 바꾸는 값이므로 `explorer` 에서 `scene` 으로 옮겼고, ShareCard 애그리거트에 해당하는 `share_card` 테이블(종류별 1행, 참조한 장면·영토 버전 기록)이 빠져 있어 추가했으며, 본문에만 있던 집계 테이블 `region_stats · rank_percentile` 을 그림에 넣었다. 상시 도전은 `quest_progress.year_month` 가 NULL이면 PK가 되지 않으므로 `'ALL'` 센티넬을 쓴다. 애그리거트 경계를 넘는 FK는 두지 않고(예: `scene.slots` 에서 `owned_item` 로), 지역·아이템·세트 같은 정의는 테이블 없이 코드로 참조한다. 삭제는 `visit` 만 물리 삭제하고 나머지는 이벤트로 따라간다.

지도 아래로 영토·도감, 탐험가 아래로 진행·인벤토리·장면이 달린다



강조 = 두 루트(지도, 탐험가). 점선 상자 = 리소스 또는 배치 집계. item_definition은 이제 DB 테이블(운영 추가).

ERD · 테이블 11개, FK는 전부 explorer로

explorer_progress 와 scene 은 1:1이라 explorer 에 합칠 수도 있지만, 변경 주체가 다르고(이벤트 vs 사용자) 낙관적 락 범위를 분리하려고 떨어뜨렸다. collected_codes · slots · props 는 조회 조건으로 쓰지 않으므로 JSON 컬럼이 맞다. 필요해지면 읽기 모델로 펼친다.

진행 순서

탐험 컨텍스트부터 세우고, 이벤트 구독자를 하나씩 붙여 가는 순서다. 각 단계는 2주 정도로 잡았고, 단계 끝마다 현재 프로토타입 화면이 그 API를 쓰도록 바꾼다.

단계	범위	산출물	프로토타입에서 옮기는 것	목표
1	카탈로그 + 탐험	Flyway V1(explorer, visit, outbox), POST /visits DELETE /visits/{code} GET /territory, CheckInPreview	지역 250개·희귀도·특산물 정의 JSON, GeoJSON	Oct 13, 2026
2	진행	V2(explorer_progress, xp_ledger, badge_earned, set_progress, quest_progress), 이벤트 핸들러 3개, 재계산 배치	XP 공식, 배지 12개, 세트 9개, 퀘스트 정의	Oct 27, 2026
3	꾸미기	V3(owned_item, scene), PUT /scene, 자동 착용 핸들러	아이템 정의(슬롯·룩·색), 픽셀 렌더러는 클라이언트에 그대로	Nov 10, 2026
4	공유	공개 프로필 /u/{handle}, OG 카드 렌더러(서버 캔버스), 스냅샷 저장·무효화	카드 4종 레이아웃	Nov 24, 2026
5	소셜	V4(friendship, feed_entry), 피드·랭킹 프로젝트, VS 비교 쿼리, 상위 % 배치	VS 비교 로직	Dec 8, 2026

1단계가 끝나면 로그인 없이 익명 탐험가로 칠하다가 공유 시점에 계정을 연결하는 흐름이 나온다. 이때 익명 explorer를 계정에 병합하는 커맨드(claimExplorer)가 필요하고, 이건 Territory가 아니라 explorer 레벨의 일이다. Kafka는 5단계까지 필요 없고, outbox 릴레이를 @Scheduled 로 돌리다가 트래픽이 생기면 바꾼다.

설계 판단과 열린 질문

가장 큰 판단은 "탐험가 한 명 = 애그리거트 하나"를 거부한 것이다. 프로토타입은 state 객체 하나에 전부 담았지만, 서버에서 그렇게 하면 착용 변경 한 번에 영토 전체를 잠그고, 세트 정의 추가 같은 운영 변경이 매번 루트를 건드린다. 대신 영토를 유일한 진실로 두고 나머지를 파생 상태로 취급하면, 어떤 규칙을 바꿔도 배치 한 번으로 전부 다시 계산할 수 있다.

두 번째는 취소의 비대칭성이다. 체크인은 언제든지 취소할 수 있지만 세트 완성·배지·퀘스트 보상은 되돌리지 않는다. 자기 신고제라 "체크했다 풀었다"로 보상을 반복 획득하는 것은 이미 멍등성(refId)으로 막히므로, 회수 로직의 복잡도를 들일 이유가 없다.

세 번째, 읽기 모델을 애그리거트로 만들지 않았다. 랭킹·피드·리캡·VS 비교는 전부 이벤트로부터 다시 만들 수 있는 투영이다.

열려 있던 질문은 다음과 같이 결정했다.

질문	결정	설계 반영
방문 날짜를 과거로 고칠 때 소급 재계산?	스트릭·월간 퀘스트만 체크인 시점 기준(소급 없음). 영토·아이템·세트·리캡은 과거 날짜 그대로 인정	<code>RegionVisited.visitedAt</code> 은 처리 시각(서버 시계). <code>Visit.visitDate</code> 는 기록용 표시 값일 뿐 진행에 영향 없음. <code>VisitEdited</code> 는 진행이 구독하지 않음
위치 검증?	지금은 없음. 모바일 진입 시 추가	<code>Visit</code> 에 <code>verification(NONE GPS)</code> VO 자리만 남김. 현재는 항상 NONE, 랭킹 필터는 나중에
아이템을 동적으로, 이슈마다 추가?	동적. 지역 특산물 외에 이슈·시즌 아이템을 운영이 수시로 추가	<code>ItemDefinition</code> 이 리소스에서 DB 테이블로 옮겨감. 지급 규칙(<code>grantRule</code> : 지역 방문 / 기간 내 체크인 / 세트 완성 / 수동)과 유효 기간을 가진다. 희귀도는 여전히 정의에 고정
세계 확장?	확장 예정	<code>Region</code> 에 <code>countryCode</code> 를 지금 넣고 <code>RegionCode</code> 를 KR-11010 처럼 국가 접두어 포함 형식으로. 정복률 집계는 국가 단위로
공동 영토?	초대코드로 같은 지도를 공유, 캐릭터는 각자	공유 지도(ExpeditionMap) 애그리거트 신설. <code>Territory</code> 의 키가 <code>explorerId</code> 에서 <code>mapId</code> 로 바뀐다(아래 추가 설명)

공유 지도 반영

모든 탐험가는 가입 시 자기 지도 하나를 가지고 시작하고(멤버 1명), 초대코드로 다른 지도에 합류할 수 있다. 지도에 속한 것과 탐험가에 속한 것을 나누면 다음과 같다.

지도 단위 (멤버 전원 공유)	탐험가 단위 (각자)
<code>Territory(mapId)</code> — 누가 체크인했는지 <code>checkedInBy</code> 기록	<code>ExplorerProgress</code> — XP·레벨·칭호·스트릭은 본인 체크인만 집계
<code>Collection(mapId)</code> — 세트 완성은 지도 기준, 세트 배경은 멤버 전원 지급	<code>QuestBoard</code> — 월간 퀘스트는 본인 기준
정복률·전설 풍경·랭킹의 영토 수	<code>Inventory</code> — 지역 아이템은 체크인한 본인에게, 세트 보상은 전원에게
<code>ExpeditionMap</code> — 이름, 초대코드, 멤버(≤ 4), 생성자	<code>Scene</code> — 캐릭터·착용은 각자. 장면에 멤버 캐릭터가 나란히 선다

공유 지도는 협력이 아니라 **경쟁**이다. 멤버는 각자 다니고, 같은 지역을 각자 체크인할 수 있다. Visit 은 (지도, 지역, 멤버) 단위로 한 행이고, 지역의 색은 가장 먼저 체크인한 멤버(선점자)의 색으로 칠해진다. 지도 안 랭킹은 멤버별 영토 수·선점 수·전설 지역 수로 매긴다. RegionVisited 이벤트는 mapId 와 explorerId 를 둘 다 실어 보내고 isFirstClaim 을 넣어 선점 보너스를 줄 수 있게 한다. 지역 아이템은 체크인한 본인이 받고, 세트 완성은 지도 기준(멤버 누가 다녀든 지역이 찼으면 완성)으로 판정해 배경을 전원에게 준다. 탈퇴하면 그 멤버의 Visit은 지도에서 삭제된다(MemberLeft → Territory가 해당 행 삭제, 선점자는 다음 체크인자로 넘어감). 본인 진행·인벤토리는 유지된다. 합류할 때 기존 개인 지도는 병합하지 않고, 지도는 여러 개 가질 수 있다.

랭킹은 두 층이고 서로 다른 집계다. 지도 안 랭킹은 멤버별로 visit 을 (map_id, checked_in_by)로 세고, 전체 랭킹과 친구 랭킹은 탐험가별로 본인이 체크인한 **중복 제거한 지역 수**를 센다(여러 지도에서 같은 지역을 체크인해도 1). 그래서 경쟁 지도에서 추가로 다닌 곳은 전체 랭킹에 그대로 반영되고, 같은 곳을 지도마다 다시 찍어 전체 랭킹을 부풀릴 수는 없다. 이를 위해 진행 컨텍스트가 explorer_region (explorer_id, region_code, first_visited_at) 읽기 모델을 유지하고, 전체 랭킹·상위 %·도감 배지("50곳 탐험")는 이 테이블을 센다. 탈퇴로 지도에서 Visit이 삭제돼도 탐험가의 explorer_region 은 남으므로 전체 랭킹은 줄지 않는다.

XP도 같은 원칙이다. 기본 XP는 탐험가당 지역당 한 번(xp_ledger.ref_id = region:{explorerId}:{code}), 선점 보너스는 지도마다(claim:{mapId}:{code}). 선점 보너스는 **준다**: 최초 선점 +10 XP, 선점자가 탈퇴해 선점이 다음 체크인자에게 넘어가면 그 사람에게도 +10 XP(ClaimTransferred 이벤트, ref_id가 다르므로 중복 없음). 떠난 사람의 보너스는 회수하지 않는다.

여전히 열려 있는 것:

- ☐ 이슈 아이템의 지급 조건을 규칙 엔진까지 같지("기간 내 특정 시·도 체크인" 정도는 이념으로 충분), 운영 화면은 언제 만들지

기획 리스크 점검

가장 큰 리스크는 두 개다. 자기 신고제 위에 경쟁과 보너스를 올리면 치팅 유인이 생기고, "소급 없음"을 영토에까지 적용하면 초기 유입의 핵심인 "예전에 간 곳 채우기"가 막힌다. 나머지는 지금 결정해 두면 값싼 것들이다.

#	리스크	왜 문제가 되나	대응
1	소급 금지의 범위	가입 직후 유저가 하는 일은 지난 여행을 채우는 것이다. 과거 날짜 입력을 막거나 의미 없게 만들면 첫 세션에서 이탈한다	소급 금지는 스트릭·월간 퀘스트에만. 영토·아이템·세트·리캡은 과거 날짜를 그대로 인정. 온보딩에서 "다녀온 곳 한번에 찍기"를 첫 화면으로

#	리스크	왜 문제가 되나	대응
2	자기 신고 + 경쟁 = 치팅 유인	선점 보너스와 지도 랭킹이 있는데 검증이 없으면 "안 가고 찍기"가 생긴다. 친구 사이라도 분쟁으로 번진다	지도 설정에 "사진 필수" 옵션, 하루 체크인 상한(예: 5곳), 지도장이 멤버 체크인에 "이의" 표시 가능. 전체 랭킹은 처음부터 "참고용"으로 약하게
3	공유 지도 규칙이 복잡함	같은 지역 각자 체크인, 선점자 색, 탈퇴 시 삭제와 선점 이전은 설명 없이 이해하기 어렵다. 탈퇴 한 번에 지도 절반이 바뀔 수 있다	지도 생성 시 규칙 3줄 안내, 탈퇴 전 "내 영토 N곳이 지도에서 사라집니다" 경고, 탈퇴 후 7일 복구 유예(소프트 삭제)
4	콘텐츠 소진	지역 250개·세트 9개는 열심히 유저가 몇 달이면 바닥난다. 이후 돌아올 이유가 월간 퀘스트뿐	이슈 아이템(결정됨)에 더해 계절 한정 세트(벚꽃·단풍), 연 1회 "재방문 도장"(영토는 그대로, 아이템 변형), 세계 확장을 두 번째 시즌 콘텐츠로
5	아이템 가치 희석	체크인마다 아이템 1개면 가방이 금방 수백 개다. 희귀함이 사라지고 가방 화면이 무거워진다	일반 아이템은 "키링"처럼 착용 효과가 작은 슬롯에 몰고, 진열은 희귀·전설·세트·이슈 아이템 중심. 즐겨찾기·최근 획득 필터. 합성·강화는 넣지 않는다
6	콜드 스타트	혼자 시작하면 비교 대상이 없고, 지도 멤버는 ≤4이라 지인 초대가 유일한 성장 경로다	초대 양쪽 보상(한정 아이템), 공개 프로필 링크가 지도 초대 링크 역할을 겨하도록(카드 보고 바로 합류), 친구 0명일 때는 "내 지역 평균 유저"와 비교
7	위치 프라이버시	공개 프로필에 방문 날짜·사진·메모가 그대로 나가면 거주지·생활 패턴이 추정된다	메모·사진은 기본 비공개, 공개 프로필은 영토 색칠과 집계만. 날짜는 월 단위로 둬글게. 공개 범위 설정(전체·친구·비공개)
8	행정구역 변경	시군구 통폐합이 실제로 일어난다(군위군 대구 편입 등). 코드가 사라지면 기존 방문 기록과 세트가 깨진다	region 카탈로그에 버전과 "대체 코드" 필드. 폐지 코드는 삭제하지 않고 속김 처리, 세트 정의는 대체 코드로 재평가
9	상호·상표	아이템 이름에 성심당·이성당·에버랜드 같은 상호가 그대로 들어가 있다	일반명사화("대전 튀김소보로", "군산 단팔빵", "용인 놀이공원 풍선"). 지역명+특산물만
10	자랑 카드 렌더링 비용	체크인·착용 변경마다 무효화하면 활발한 유저 한 명이 하루 수십 번 서버 렌더링을 일으킨다	무효화만 하고 렌더링은 공유 링크가 실제로 열릴 때(lazy). 최소 TTL 10분

#	리스크	왜 문제가 되나	대응
11	운영 부담	이슈 아이템을 "운영이 수시로" 만들려면 그림·규칙·기간을 매번 손으로 넣어야 한다	아이템은 엔진 룩(형태+색)의 조합이므로 운영 화면에서 폼으로 만들 수 있게. 지금 규칙은 3종(기간 내 체크인 / 특정 시·도 / 세트 완성)만

위 대응은 전부 채택했고, 설계에는 다음처럼 들어간다.

항목	반영 위치	단계
소급 범위: 스트릭·퀘스트만 체크인 시점, 나머지는 과거 날짜 인정	설계 판단 표, Territory/QuestBoard 불변식	1
하루 상한 5건(지도별) + 가입 72h 온보딩 예외	Territory 불변식, MapSettings	1
사진 필수 옵션, 지도장 이의(disputed), 3줄 규칙 안내, 탈퇴 7일 유예(hidden_at)	ExpeditionMap, visit 테이블	3
전체 랭킹은 참고용(상위 % 표시), 경쟁은 지도 내에서만	rank_percentile, 소셜 컨텍스트	4
계절 세트, 재방문 도장(같은 지역 2회차부터 카운트)	Collection, set_progress	2
키링 슬롯 3개 + 즐겨찾기 플래그	Inventory(owned_item.favorite)	2
초대 보상(초대자·피초대자 모두), 프로필 공개 링크	Social, ShareCard	4
PrivacySettings(지도 공개 범위) + 공유 카드 날짜는 월 단위	Sharing, scene/share_card	4
Region version · replacedBy · retiredAt 로 행정구역 개편 대응	Region Catalog 참조 데이터	1
상호·브랜드 아이템명은 일반명사화(예: ○○야구장 → 야구장 티켓)	item_definition 운영 규칙	1
공유 카드 lazy 렌더, 캐시 TTL 10분	ShareCard 인프라	4
이벤트 아이템 운영 폼 POST /admin/items	Wardrobe api	2